

AI Software Assessment — Completed Exercises (Python)

Worked solutions for Exercises 1-20 of the WeThinkCode_ AI Software course (Module 2 "Using AI for everyday work" and Module 3 "Learning with AI"). Each exercise has its own folder containing a **submission document** (named after the exercise, with the exercise description at the top and a completed "Submission for Exercise" section), plus any code and tests.

Starter code source: `../ai-code-exercises/use-cases/` · Language: **Python** · Verified with `python3 -m unittest / pytest`.

FastAPI exercises (17-20) need extra packages — install with `pip install -r requirements.txt`.

#	Exercise	Deliverables	Tests
1	Knowing where to start	analysis doc	— (doc only)
2	Codebase Exploration Challenge	journal + presentation doc	—
3	Algorithm Deconstruction Challenge	deconstruction doc	—
4	Code Documentation	<code>task_priority_documented.py</code> + doc	syntax OK
5	API Documentation	endpoint/OpenAPI/guide doc	—
6	README & User Guide	<code>README.md</code> + doc	—
7	Error Diagnosis Challenge	diagnosis doc	—
8	Performance Optimization	<code>inventory_analysis_optimized.py</code> + <code>benchmark.py</code> + doc	~630-1385× faster, output verified identical
9	AI Solution Verification	<code>merge_sort.py</code> + <code>test_merge_sort.py</code> + doc	4 pass
10	Using AI to help with testing	<code>task_priority.py</code> + <code>models.py</code> + <code>test_task_priority.py</code> + doc	12 pass
11	Understanding What to Change	analysis doc	—
12	Function Decomposition	<code>sales_report.py</code> + <code>test_sales_report.py</code> + doc	8 pass (behaviour preserved)

#	Exercise	Deliverables	Tests
13	Code Readability	<code>calculator.py</code> , <code>discount.py</code> + tests + doc	discount 6 pass ; calculator 3/4 (1 pre-existing starter mismatch, documented)
14	Design Pattern (Factory)	<code>database_connection.py</code> + <code>test_database_connection.py</code> + doc	8 pass (behaviour preserved)

Module 3 — Learning with AI

#	Exercise	Deliverables	Tests
15	Deepen current language (Python)	journal doc	— (doc only)
16	Learning a new language (Go)	<code>main.go</code> mini-project + doc	not compiled (Go not installed)
17	Getting Started with FastAPI	to-do CRUD <code>app.py</code> + <code>test_app.py</code> + doc	6 pass
18	Contextual Learning (JWT auth)	<code>app.py</code> + <code>test_app.py</code> + doc	6 pass
19	Documentation Navigation (blog API)	<code>app.py</code> + <code>test_app.py</code> + doc	7 pass
20	FastAPI Code Patterns (+ logging feature)	<code>app.py</code> + <code>test_app.py</code> + doc	6 pass

Notes on findings

- **Ex8:** the original's real bottleneck was an $O(R)$ duplicate scan *inside* the inner loop, not just the $O(n^2)$ loops — the optimised sort+bisect version is 100s-to-1000s× faster and returns identical pairs.
- **Ex10:** first test run was flaky on due-date buckets due to `timedelta.days` truncation at day boundaries; fixed by using hour-based mid-bucket offsets.
- **Ex13:** the provided compound-interest test contradicts the provided code (original returns 2239.52 vs the test's 2234.51) — flagged rather than silently "fixed", since a documentation task shouldn't change behaviour.
- **Ex16:** the Go mini-project is written idiomatically but not compiled here (no Go toolchain in the sandbox); everything else is executed and verified.
- **Ex20:** the advanced sample was adapted from SQLAlchemy-async to an in-memory store so it runs without external services, keeping all the patterns (generic repository, DI, middleware, RBAC, lifespan) intact; the sample's RBAC decorator was converted to the idiomatic FastAPI dependency form.

Running the tests

```
cd <exercise-folder>  
python3 -m unittest -v
```